ALLIANCE UNIVERSITY

Private University Estd. in Karnataka State by Act No . 34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Department of Computer Science and Engineering

B. Tech. CSE (AIML)

Software Engineering & Agile Development

6CS1016

CASE STUDY ANALYSIS REPORT

Tittle: CrowdStrike 2024 Global Outage

Under the Supervision of

Dr. Dr Sengottaiyan N, Professor.

Submitted by

Pathan Mohammed Akram Khan - 2023BCSE07AED041

CASE STUDY ANALYSIS REPORT
CrowdStrike 2024 Global Outage – SEAD

ABSTRACT

The CrowdStrike global outage of July 2024 is a critical case demonstrating how failures in Agile governance and DevOps release management can disrupt worldwide digital infrastructure. The incident was caused by a defective kernel-level content update that triggered Blue Screen of Death (BSOD) errors across millions of Windows systems, impacting aviation, healthcare, banking, and government sectors.

This case study analyzes the failure from the perspective of Software Engineering and Agile Development (SEAD). It examines weaknesses in requirement validation, regression testing, CI/CD pipeline controls, risk assessment, and change management practices. Agile ceremonies such as sprint planning and review failed to identify deployment risks, while the absence of canary release and rollback mechanisms allowed the defect to propagate globally.

The study applies SDLC principles, Agile Scrum practices, and risk management frameworks to evaluate the incident. Recommendations include staged deployment, automated rollback, enhanced kernel testing, and formal CAB governance. The findings highlight the necessity of disciplined Agile processes combined with rigorous software engineering practices to ensure reliability and ethical responsibility in safety-critical systems.

1. INTRODUCTION

The modern software ecosystem is fundamentally driven by Agile development methodologies and automated DevOps pipelines. While these approaches enable rapid innovation and continuous delivery, they also introduce systemic risks when engineering discipline and quality assurance are inadequate. The CrowdStrike global outage of July 2024 represents a significant real-world example of how a single defective update can escalate into a worldwide technological crisis.

This report critically examines the incident through the lens of Software Engineering principles, Agile process governance, DevOps release management, Quality Assurance practices, and professional ethics. The objective is to derive structured lessons that can guide future large-scale software deployments.

2. OBJECTIVES OF THE STUDY

The primary objectives of this case study are:

- To critically analyze the software engineering failures that led to the outage
- To identify deficiencies in Agile planning and release governance
- To evaluate weaknesses in the DevOps CI/CD pipeline
- To propose a structured preventive and corrective framework
- To assess ethical and professional responsibilities of software engineers

3. PROBLEM STATEMENT

Modern cybersecurity platforms rely on frequent automated updates to protect endpoints from emerging threats. However, improper validation of critical kernel-level updates can lead to catastrophic system failures.
In the CrowdStrike case, the existing deployment process failed due to lack of staged rollout, insufficient regression testing, and absence of automated rollback mechanisms. This resulted in a defective update being deployed simultaneously across global systems.
Stakeholders involved include enterprise customers, IT administrators, end users, healthcare institutions, airlines, and financial organizations whose operations depend on stable software infrastructure.

4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

The Falcon platform should:

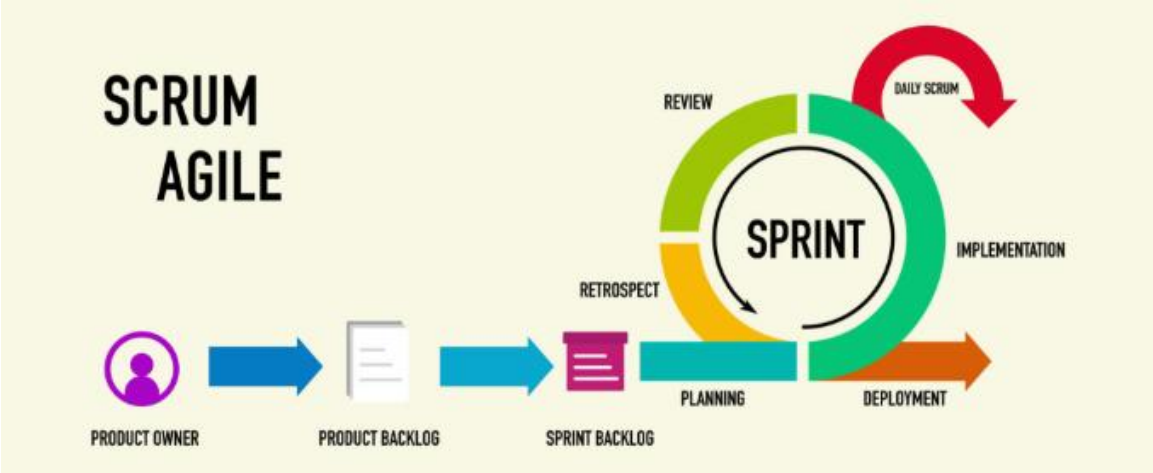
- Validate update packages before deployment
- Monitor system behavior post-update
- Provide rollback functionality for failed updates
- Maintain real-time threat detection and logging

4.2 Non-Functional Requirements

- **Performance:** Updates must not degrade OS stability
- **Security:** Kernel drivers must be validated against malicious or malformed inputs
- **Scalability:** System should support phased global deployment
- **Reliability:** Automatic recovery and rollback must be supported
- **Usability:** Administrators should receive clear update failure alerts

5. SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) MODEL USED

The CrowdStrike system primarily follows an Agile SDLC model integrated with DevOps CI/CD pipelines. Agile is chosen due to the need for continuous threat intelligence updates and rapid feature releases.
Compared to the Waterfall model, Agile allows iterative development, faster feedback, and adaptive requirement handling. However, this case shows that Agile must be supported by strict quality gates and staged validation when dealing with critical kernel-level software.
The absence of formal phase validation and risk analysis weakened the effectiveness of the Agile lifecycle in this incident.



This diagram illustrates the iterative Agile Software Development Life Cycle where the process progresses through repeated cycles of planning, development, testing, deployment, and feedback. Each iteration (sprint) delivers incremental functionality, allowing teams to quickly identify defects, incorporate stakeholder feedback, and adapt to changing requirements. This continuous loop ensures rapid release cycles while maintaining software quality and risk control, which is especially critical for systems requiring frequent updates such as cybersecurity platforms.

6. AGILE METHODOLOGY APPLIED (SCRUM)

The development lifecycle followed Scrum practices with continuous sprints.

Product Backlog

Security updates, threat signatures, kernel driver improvements.

Sprint Planning

Tasks related to malware detection and performance optimization were prioritized; however, risk evaluation for deployment was insufficient.

Sprint Duration

Short release cycles with frequent content updates.

Daily Stand-up

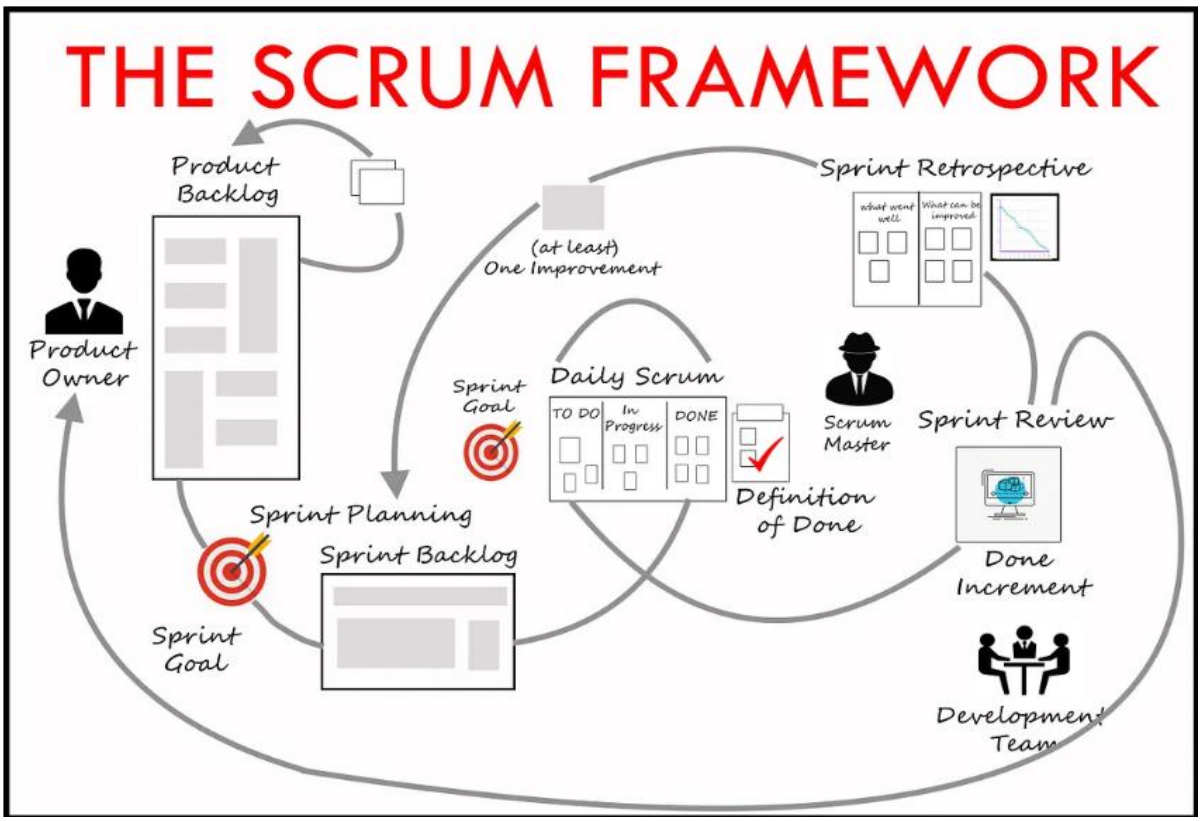
Focused on development progress but lacked cross-functional risk discussion.

Sprint Review & Retrospective

Feedback cycles existed but did not emphasize deployment safety metrics.

Agile Artifacts

- User Stories: Update stability and threat detection improvements
- Story Points: Estimated effort for update modules
- Velocity: High delivery speed prioritized over reliability
- Burndown Chart: Used for sprint progress



Shows how Scrum manages iterative development through backlog prioritization, sprint execution, daily stand-ups, and continuous review.

7. ORGANIZATION BACKGROUND

CrowdStrike is a globally recognized cybersecurity organization providing endpoint protection through its Falcon platform. The platform operates with kernel-level privileges to detect advanced threats in real time. Due to this deep system integration, the software demands extremely high reliability, rigorous testing, and carefully controlled update mechanisms.

The Falcon platform distributes frequent content and sensor updates to millions of endpoints worldwide through automated delivery infrastructure. This architecture, while efficient, requires strong safeguards to prevent cascading failures.

8. INCIDENT DESCRIPTION

On 19 July 2024, a routine content update released through CrowdStrike's automated pipeline triggered widespread Blue Screen of Death (BSOD) errors on Windows systems. The failure rapidly propagated across industries including aviation, healthcare, banking, government services, and media organizations.

Restoration required manual remediation on individual devices, resulting in prolonged downtime, operational disruption, and significant economic loss. The incident highlighted the critical dependency of modern infrastructure on secure and reliable software updates.

9. TECHNICAL ROOT CAUSE

Technical investigation revealed that the CsAgent.sys kernel driver processed a malformed channel file without adequate memory validation. The defect bypassed existing automated tests due to insufficient kernel-level test coverage and inadequate regression scenarios.

Key technical failures included:

- Absence of defensive input validation
- Inadequate simulation of real-world OS environments
- Lack of staged verification before global deployment
- Over-reliance on automation without human oversight

10. DEVOPS PIPELINE ANALYSIS

The analysis of the CI/CD pipeline identified several structural weaknesses:

- Non-existence of canary or phased release strategy
- Simultaneous global deployment without segmentation
- No automated rollback or kill-switch mechanism
- Limited regression and compatibility testing
- Insufficient real-time monitoring and observability

These gaps demonstrate violation of core DevOps principles such as progressive delivery, feedback loops, and resilience engineering.

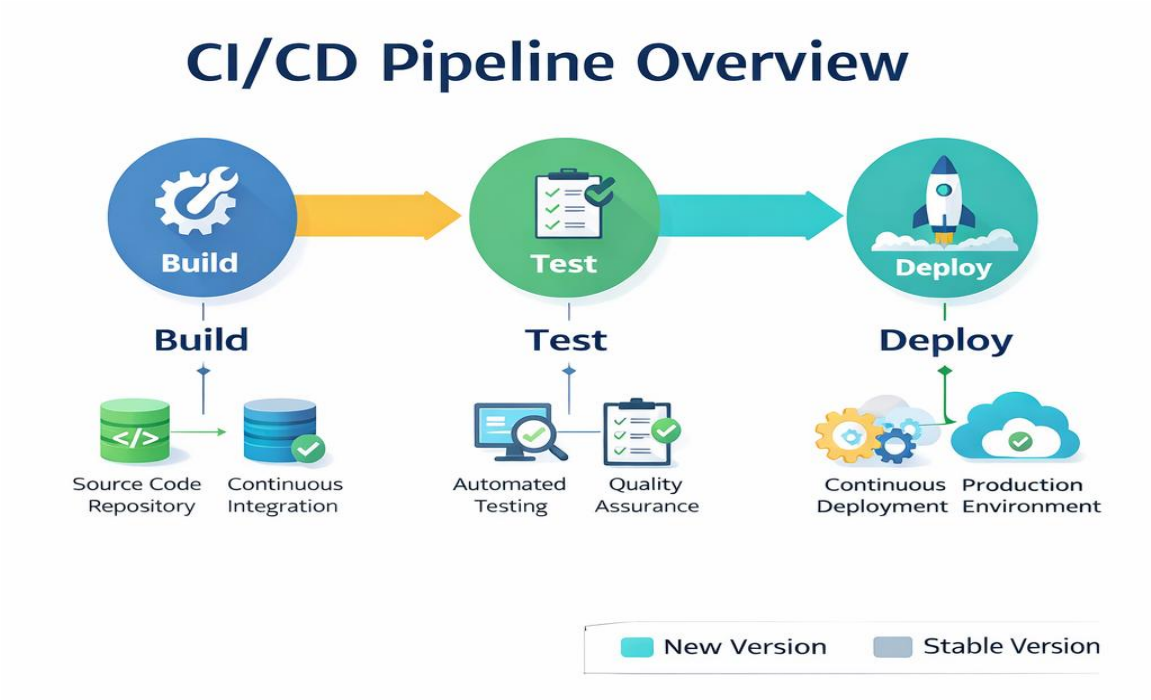


Figure 1: CI/CD Pipeline Illustrating Build–Test–Deploy Stages in DevOps

11. AGILE PROCESS ISSUES

From an Agile governance perspective, the following deficiencies were observed:

- Sprint planning lacked formal risk assessment
- "Definition of Done" excluded stability and reliability criteria
- Release management bypassed Change Advisory Board (CAB)
- Poor communication between development, QA, and operations teams

This reflects an imbalance between development velocity and engineering discipline.

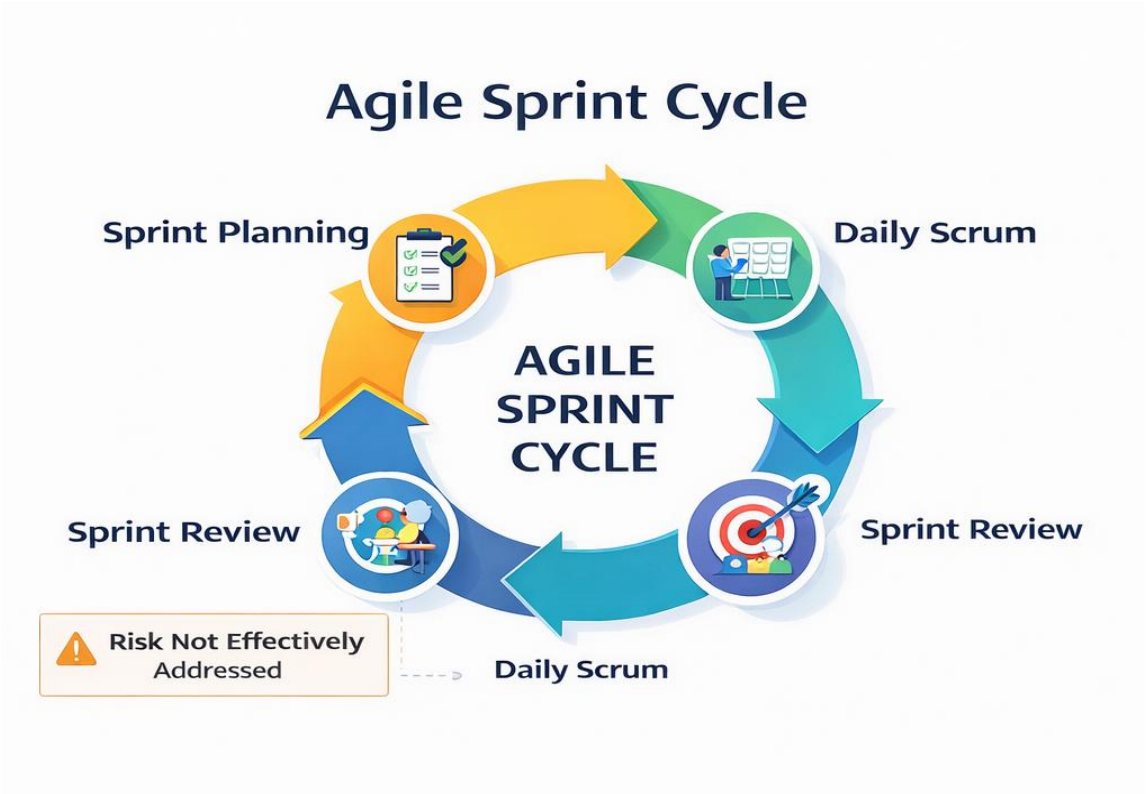


Figure 2Agile Sprint Cycle Showing Planning, Daily Scrum, Review and Retrospective

12. SOFTWARE ENGINEERING VIOLATIONS

The incident contradicted fundamental Software Engineering best practices:

- Incomplete regression and compatibility testing
- Weak change management controls
- Absence of fault-tolerant design
- Lack of defensive programming at kernel level
- Insufficient validation of critical system components

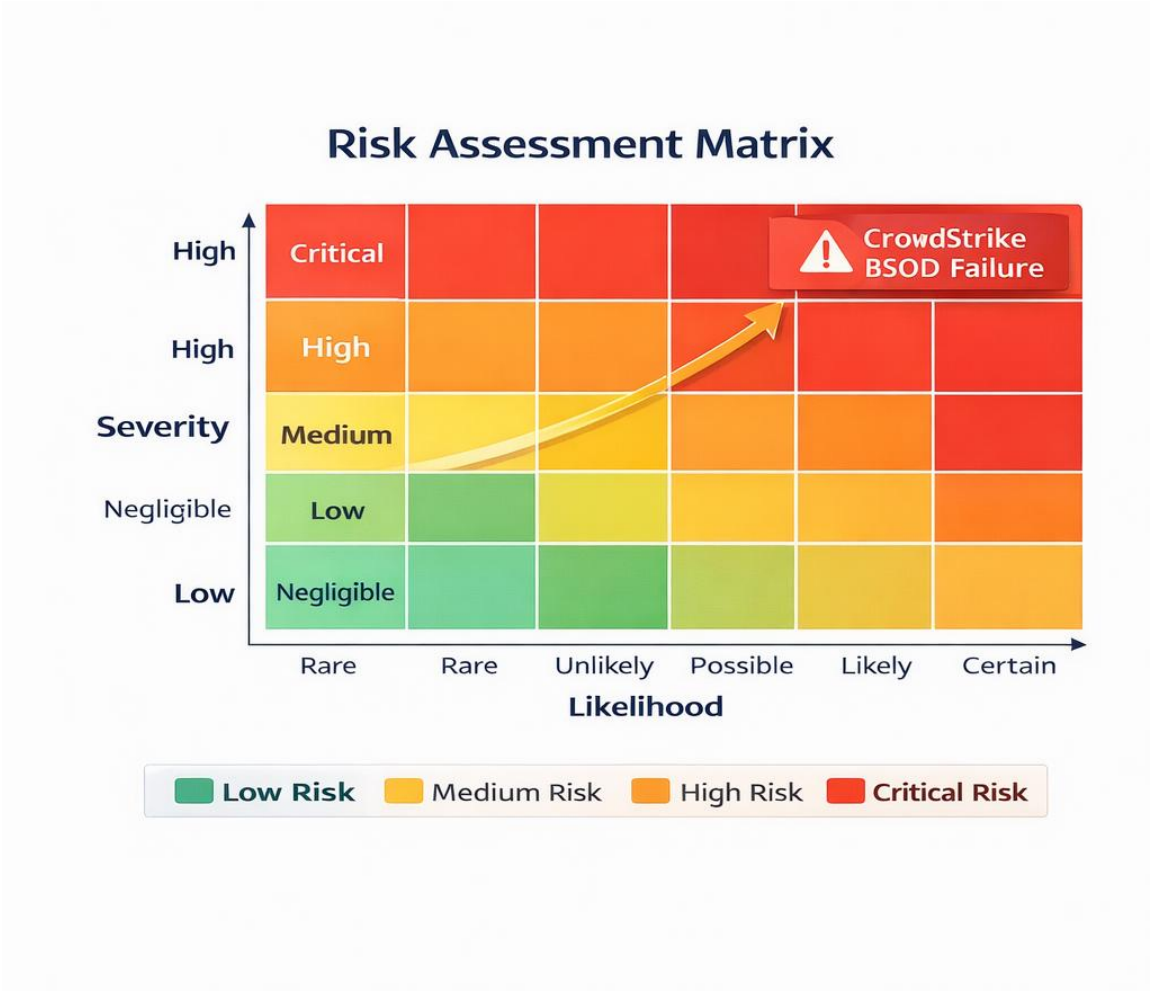


Figure 3Risk Assessment Matrix Indicating Critical Severity of Kernel Update Failure

13. RISK ANALYSIS

13.1 Technical Risks

- Kernel compatibility failures
- Incorrect update configuration
- Performance degradation

13.2 Requirement Risks

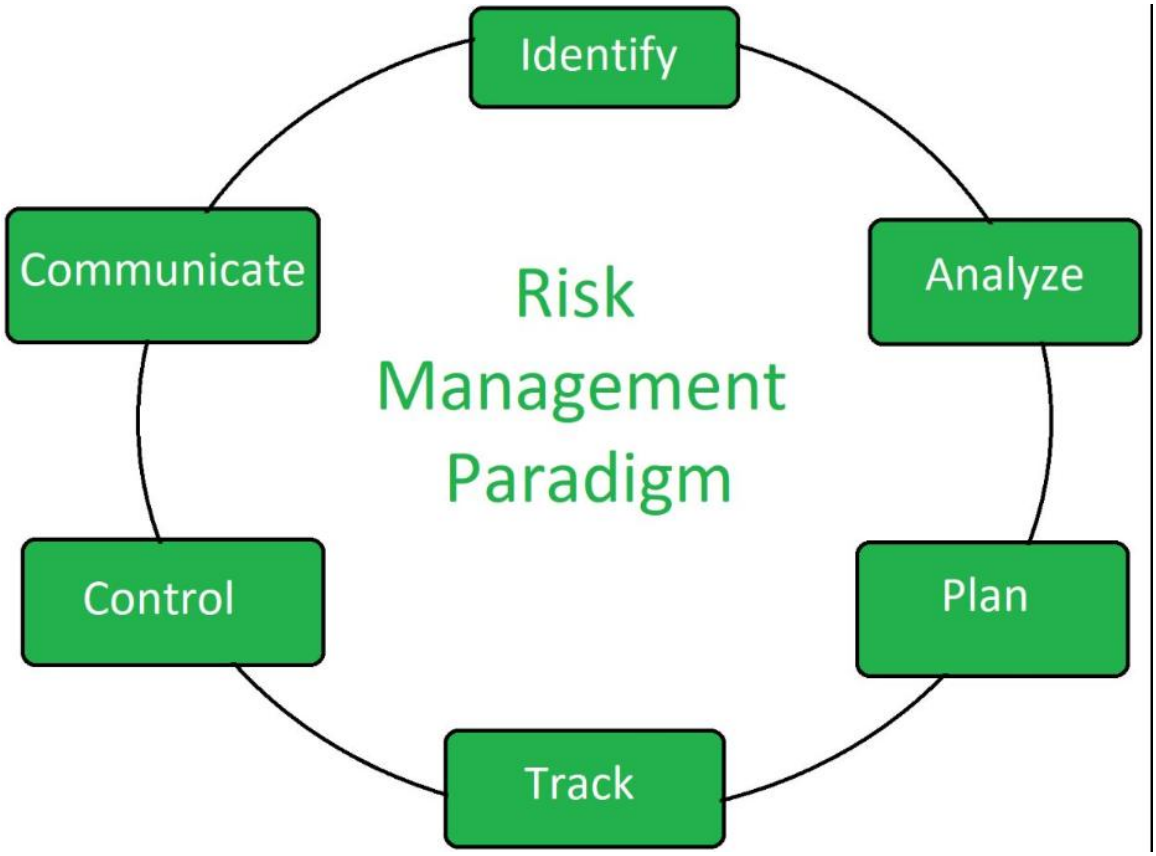
- Frequent updates causing instability
- Changing threat detection requirements

13.3 Resource Risks

- Over-reliance on automated deployment
- Insufficient manual validation resources

13.4Mitigation Plan

- Canary deployment model
- Automated rollback mechanisms
- Enhanced monitoring and anomaly detection
- Dedicated kernel test environments



Depicts the systematic process of identifying, analyzing, mitigating, and monitoring risks during software development and deployment.

14. IMPACT ANALYSIS

The outage affected millions of endpoints globally, leading to:

- Flight cancellations and airport disruptions
- Delays in healthcare services
- Interruptions in banking and payment systems
- Loss of productivity and financial damage
- Severe reputational impact on the organization

The event demonstrated the societal scale of software failures in critical infrastructure.

15. ETHICAL CONSIDERATIONS

Software engineers have a professional obligation to prioritize public safety, reliability, and transparency over rapid feature delivery. Ethical practice requires:

- Responsible disclosure of risks
- Adequate testing before release
- Accountability for failures
- Protection of stakeholders from harm

16. RECOMMENDATIONS

To prevent similar incidents, the following measures are proposed:

- Implement 1–5–10–50–100% canary deployment model
- Establish automated rollback within minutes
- Develop dedicated kernel testing laboratories
- Adopt chaos engineering practices

- Enforce formal CAB and release governance

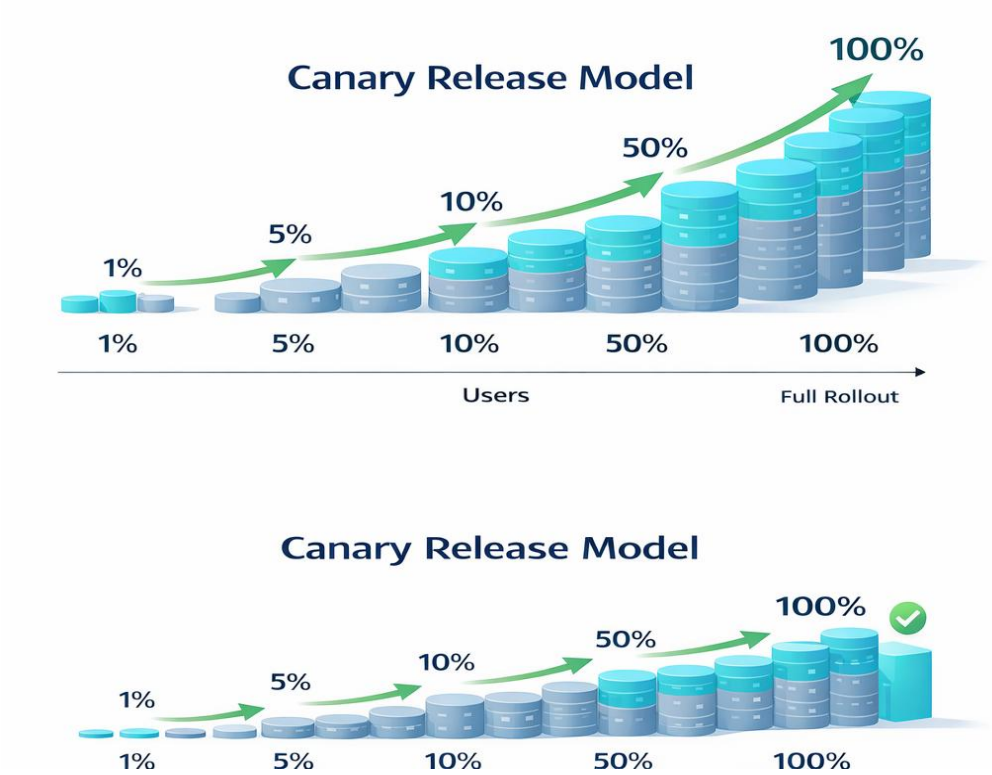


Figure 4Canary Release Strategy (1–5–10–50–100%) for Progressive Deployment

17. IMPLEMENTATION PLAN

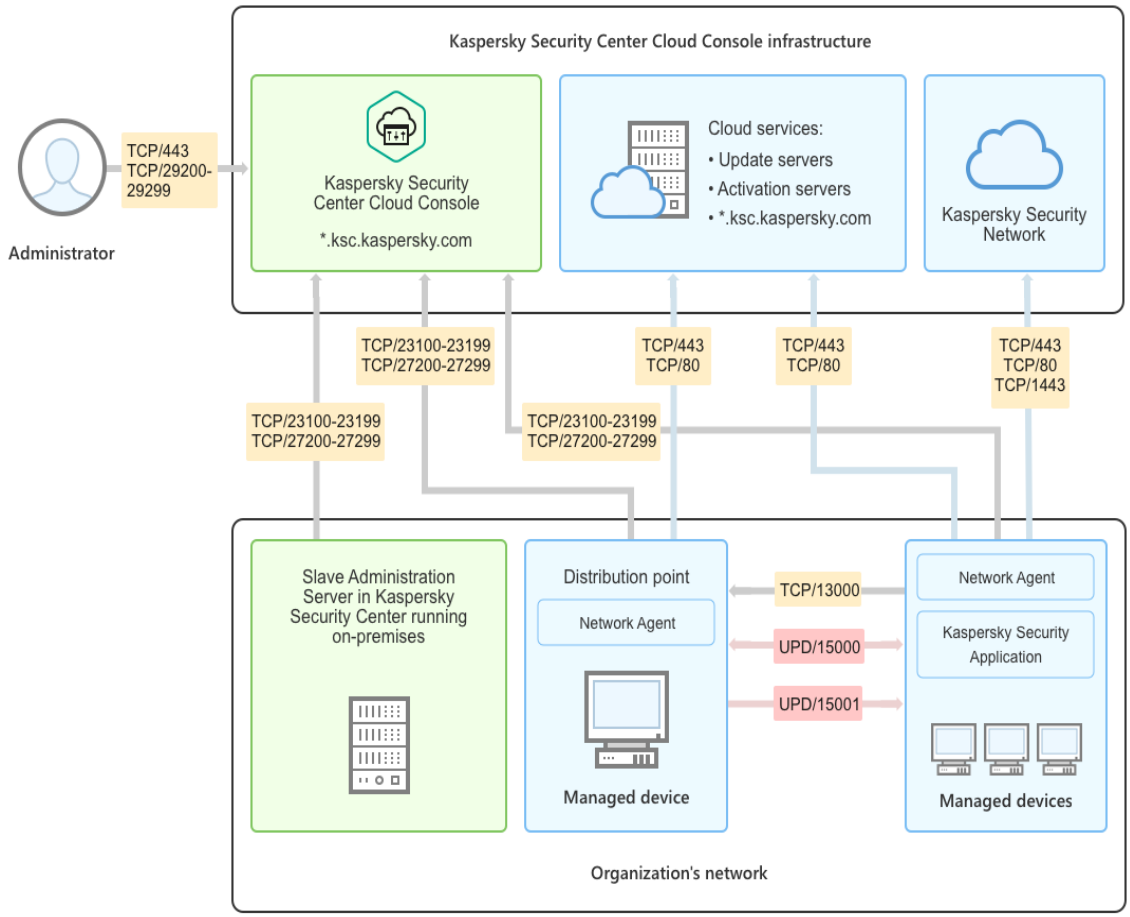
A phased roadmap is required:

- Short Term: Emergency controls, monitoring, rollback
- Medium Term: Pipeline redesign, test automation
- Long Term: Cultural transformation and reliability engineering

Dedicated QA and DevOps governance teams must oversee execution.

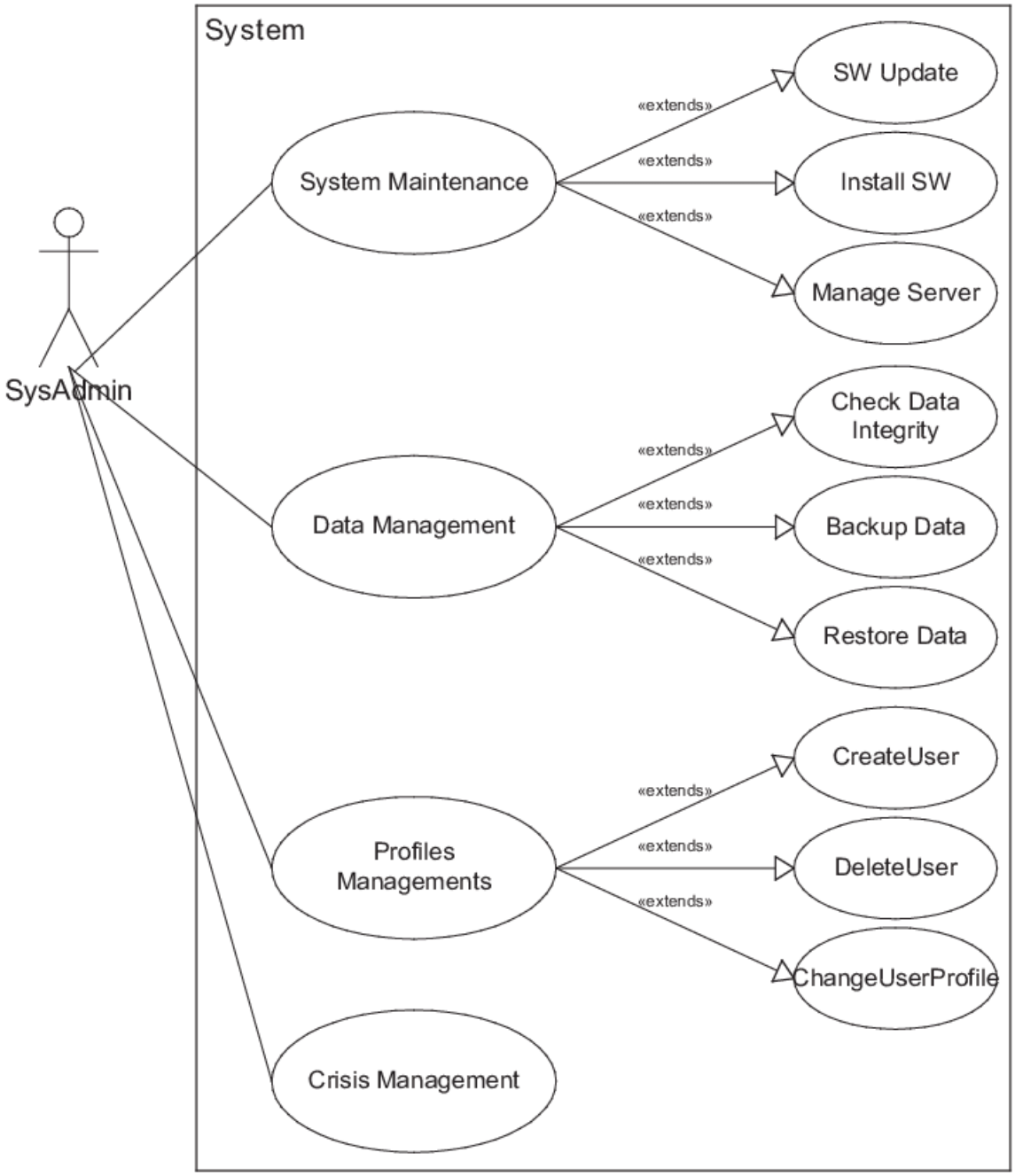
18. SYSTEM DESIGN

18.1 Architecture Diagram:



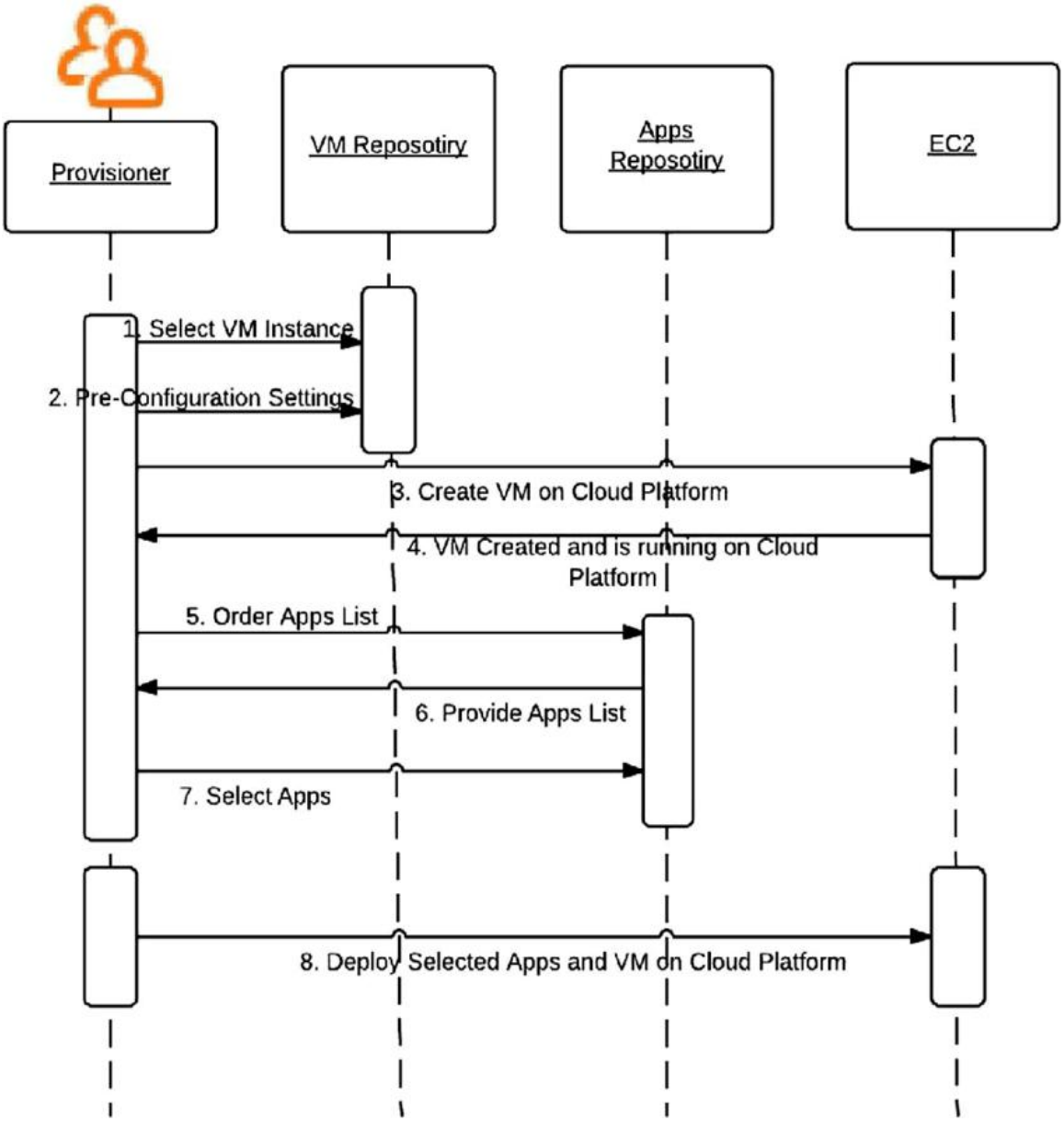
The architecture consists of endpoint agents, cloud analytics engine, and centralized management console. Updates are delivered through CDN pipelines, requiring strict validation controls to avoid cascading failures.

18.2 Use Case Diagram



This use case diagram illustrates how administrators deploy updates, the system validates and installs them, monitors performance, and triggers rollback in case of failure. It highlights interactions between the admin, endpoint agent, and cloud platform during secure update management.

18.3. Sequence Diagram – Update Deployment Flow



Depicts interaction between admin console, cloud server, and endpoint agent during update deployment and failure handling process.

19. IMPLEMENTATION DETAILS

19.1 Technologies Used

- Endpoint Agent: C/C++ Kernel Driver
- Cloud Platform: Distributed cloud infrastructure
- CI/CD Tools: Automated build and deployment pipeline
- Monitoring Tools: Telemetry and log analytics systems

19.2 Tools Used

- Version Control: Git
- Issue Tracking: Jira
- Deployment Automation: CI/CD pipelines

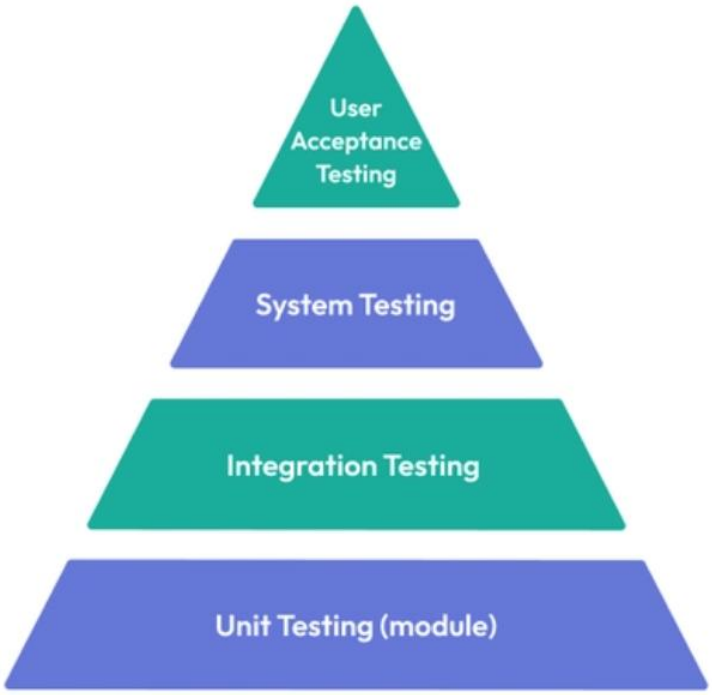
19.3 Code Structure Overview

Modular driver components interact with OS kernel, while cloud services manage update distribution and threat analytics.

20. TESTING STRATEGY

The system should incorporate multiple levels of testing:

- **Unit Testing:** Validation of kernel modules and update routines
- **Integration Testing:** Interaction between driver, OS, and cloud platform
- **User Acceptance Testing:** Deployment validation in controlled environments
- **Agile Testing Approach:** Continuous testing integrated within each sprint cycle
- **Defect Tracking:** Real-time logging and issue resolution workflow



Highlights layered **testing strategy** where unit tests form the base, followed by integration and system-level validation for reliable software releases.

21. CONCLUSION

The CrowdStrike outage illustrates that Agile and DevOps must operate within strong Software Engineering discipline. Speed without quality introduces unacceptable systemic risk. Sustainable software delivery requires balance between innovation, process control, and ethical responsibility.

22. LEARNING OUTCOMES

This case study enabled understanding of:

- SDLC and quality assurance integration
- Agile governance in critical systems
- DevOps risk management
- Professional ethics in software engineering

23. REFERENCES

- Kim et al., *The DevOps Handbook*
- Schwaber & Sutherland, *Scrum Guide*
- ISO/IEC 25010 Software Quality Model
- Industry analysis reports, 2024